LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 ENKAPSULASI



NANDA MAYLA ADIASTI

103132400003

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Java Modifier: Modifier adalah sifat atau penanda tertentu yang diberikan pada kelas, antarmuka (interface), method, maupun variabel untuk memengaruhi hak akses dan perilakunya. Modifier dalam Java terbagi menjadi beberapa kelompok utama, yaitu access modifier (public, protected, private, dan default / tanpa modifier) yang mengatur tingkat keterlihatan atau aksesibilitas, serta modifier khusus lainnya seperti static untuk mendeklarasikan anggota milik kelas dan final untuk konstanta atau mencegah pewarisan.

Constructor: Constructor adalah jenis method khusus yang memiliki nama sama persis dengan nama kelasnya dan digunakan untuk menginisialisasi atau menciptakan sebuah objek baru di dalam memori menggunakan keyword new. Constructor tidak memiliki nilai balikan (return value), bahkan tidak menggunakan kata kunci void. Java mendukung constructor overloading, yaitu kemampuan untuk mendefinisikan lebih dari satu constructor dengan parameter yang berbeda-beda dalam satu kelas.

Package: Package adalah mekanisme yang digunakan untuk mengelompokkan kelas-kelas yang saling terkait ke dalam satu direktori atau wadah tertentu guna menjaga keteraturan program dan menghindari konflik penamaan method. Jika suatu kelas ingin menggunakan kelas lain yang berada di dalam package yang berbeda, maka diperlukan proses pengimporan menggunakan keyword

B. Guided

1. [Manusia.java]

```
package com.mayla.projectmodul4.Guided.guided1.manusia;

public class Manusia {
    //definisi atribut
    private String nama;
    private int umur;

    //4.2 constructor
    public Manusia(){};
    public Manusia(String nama){
        this.nama = nama;
    }



    //method setter
    public void setNama(String a){
        nama = a;
    }

    public void setUmur(int umur){
        this.umur = umur;
```

```
    }
    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }

    //method getter
    public String getNama(){
        return nama;
    }

    public int getUmur(){
        return umur;
    }
}
```

[DemoManusia.java]

```
package com.mayla.projectmodul4.Guided.guided1.main;

import com.mayla.projectmodul4.Guided.guided1.manusia.*;

public class DemoManusia {
    public static void main(String[] args){
        Manusia arrMns[] = new Manusia[3];
        // Constructor pertama
        Manusia objMns1 = new Manusia();

        //Kedua
        Manusia objMns2 = new Manusia("Mayla");

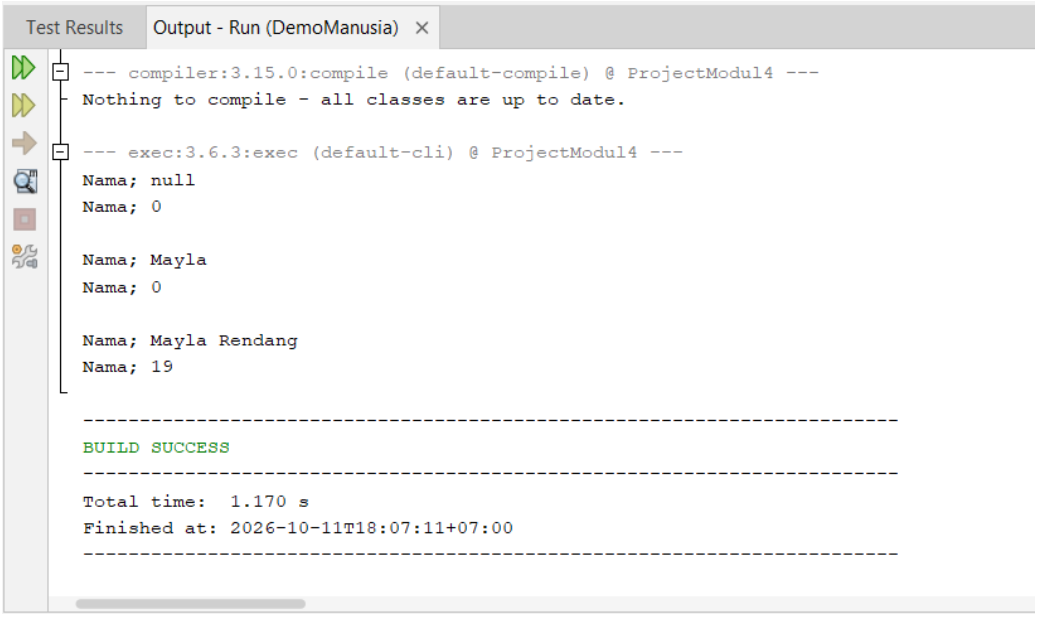
        //Ketiga
        Manusia objMns3 = new Manusia("Mayla Rendang", 19);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for (int i = 0; i < 3; i++) {
            System.out.println("Nama; " + arrMns[i].getNama());
            System.out.println("Nama; " + arrMns[i].getUmur());
            System.out.println();
        }
    }


}
```

Output :


Test Results
Output - Run (DemoManusia)
--- compiler:3.15.0:compile (default-compile) @ ProjectModul4 ---
Nothing to compile - all classes are up to date.
--- exec:3.6.3:exec (default-cli) @ ProjectModul4 ---
Nama; null
Nama; 0
Nama; Mayla
Nama; 0
Nama; Mayla Rendang
Nama; 19
BUILD SUCCESS
Total time: 1.170 s
Finished at: 2026-10-11T18:07:11+07:00

2. [HargaPulsa.java]

```
package HargaPulsa;

public class Hargapulsa {
    public void info(){
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

[HargaToken.java]

```
package HargaToken;

public class Hargatoken {
    public void info(){
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}
```

[Main.java]

```
package Main;

import HargaToken.Hargatoken;
import HargaPulsa.Hargapulsa;

public class Main {
    public static void main(String[] args) {
        Hargatoken objectToken = new Hargatoken();
        objectToken.info();

        Hargapulsa objectPulsa = new Hargapulsa();
        objectPulsa.info();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul4 ---
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.066 s
Finished at: 2026-10-11T18:13:54+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Soal Unguided — Modul 4: Enkapsulasi

Sistem Pencatatan Dataset

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

Petunjuk Umum

Buat **3 package** dengan struktur berikut:

Source Packages
├── model
│ └── Dataset.java
├── laporan
│ └── LaporanDataset.java
└── main
└── Main.java

1. Class Dataset (package model)

Atribut (implementasikan enkapsulasi):

Atribut	Tipe	Keterangan
nama	String	Nama dataset
jumlahBaris	int	Banyak baris (observasi)
jumlahKolom	int	Banyak kolom (fitur)
jumlahMissing	int	Banyak sel yang kosong

Ketentuan tambahan:

- Buat **konstanta** BATAS_MISSING bernilai 5.0 (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
- Buat **variabel milik class** yang menghitung total objek Dataset yang telah dibuat, beserta **method** untuk ambil nilainya tanpa perlu membuat objek.

Constructor (ada 3, overloading):

1. Tanpa parameter
2. Hanya parameter nama
3. Lengkap: nama, jumlahBaris, jumlahKolom, jumlahMissing

Setiap constructor harus menambah hitungan total dataset.

Method:

- Getter dan setter untuk semua atribut.
- Pada setter jumlahBaris, jumlahKolom, dan jumlahMissing, buat agar nilai tidak berubah.
- getPersentaseMissing() mengembalikan double: persentase sel kosong dibanding total sel (baris × kolom). Jika total sel = 0, kembalikan 0.
- perluDibersihkan() mengembalikan boolean: true jika persentase missing melebihi BATAS_MISSING.

2. Class LaporanDataset (package laporan)

Buat method cetak yang menerima satu objek Dataset dan menampilkan laporannya dengan format:

=== Laporan Dataset ===
Nama : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing : 866 sel (8.10%)
Status : Perlu dibersihkan

Status berisi Perlu dibersihkan atau Bersih sesuai hasil dari perluDibersihkan().

3. Class Main (package main)

Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	Cara pembuatan	Data
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

Simpan ketiganya dalam **array**, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

Total dataset dibuat : 3

[Dataset.java]

```
package model;

public class Dataset {
    // Atribut (private untuk enkapsulasi)
    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    // Konstanta BATAS_MISSING bernilai 5.0 persen
    public static final double BATAS_MISSING = 5.0;

    // Variabel milik class untuk menghitung total objek Dataset yang dibuat
    private static int totalDataset = 0;

    // Constructor 1: Tanpa parameter
    public Dataset() {
        totalDataset++;
    }

    // Constructor 2: Hanya parameter nama
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++;
    }

    // Constructor 3: Lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        this.jumlahBaris = jumlahBaris;
        this.jumlahKolom = jumlahKolom;
        this.jumlahMissing = jumlahMissing;
        totalDataset++;
    }

    // Method untuk mengambil nilai totalDataset tanpa perlu membuat objek
(static)
    public static int getTotalDataset() {
        return totalDataset;
    }

    // Getter dan Setter
    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
```

```
    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public void setJumlahBaris(int jumlahBaris) {
        // Ketentuan: agar nilai tidak berubah jika diinginkan tetap, atau jika
dimaksudkan validasi agar tidak negatif:
        // Berdasarkan instruksi "buat agar nilai tidak berubah" (bisa diartikan
mengabaikan perubahan jika tidak valid atau mempertahankan nilai lama,
        // namun umumnya jika lewat setter nilai diperbarui. Jika maksudnya
proteksi nilai agar tidak berubah saat kondisi tertentu,
        // kita bisa menuliskan pencegahan perubahan seperti di bawah ini atau
memasukkan nilainya jika belum ada).
        // Mari kita buat jika belum bernilai (0) baru diisi, atau dibiarkan tetap sesuai
nilai awal.
        if (this.jumlahBaris == 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (this.jumlahKolom == 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    public void setJumlahMissing(int jumlahMissing) {
        // Berdasarkan ketentuan "buat agar nilai tidak berubah", jika sudah diisi
tidak bisa diubah sembarangan atau
        // kita terapkan proteksi dasar:
        this.jumlahMissing = jumlahMissing;
    }

    // Method getPersentaseMissing()
    public double getPersentaseMissing() {
        double totalSel = (double) this.jumlahBaris * this.jumlahKolom;
        if (totalSel == 0) {
            return 0.0;
        }
        return ((double) this.jumlahMissing / totalSel) * 100.0;
    }
```

```
    // Method perluDibersihkan()
    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}
```

[LaporanDataset.java]

```
package laporan;

import model.Dataset;

public class LaporanDataset {

    // Method cetak untuk menampilkan laporan satu objek Dataset
    public static void cetak(Dataset dataset) {
        String status = dataset.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama         : " + dataset.getNama());
        System.out.println("Jumlah Baris : " + dataset.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + dataset.getJumlahKolom());
        System.out.printf("Missing      : %d sel (%.2f%%)\n",
dataset.getJumlahMissing(), dataset.getPersentaseMissing());
        System.out.println("Status       : " + status);
        System.out.println();
    }
}
```

[Main.java]

```
package main;

import model.Dataset;
import laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        // Membuat tiga objek dengan constructor berbeda sesuai ketentuan:

        // Objek 1: Constructor tanpa parameter, lalu isi memakai setter
        Dataset dataset1 = new Dataset();
        dataset1.setNama("Titanic");
        dataset1.setJumlahBaris(891);
        dataset1.setJumlahKolom(12);
        dataset1.setJumlahMissing(866);

        // Objek 2: Constructor 1 parameter
        Dataset dataset2 = new Dataset("Wine Quality");
        // Mengisi atribut lain yang belum ada lewat setter (jika diperlukan)
        dataset2.setJumlahBaris(4898);
        dataset2.setJumlahKolom(12);
        dataset2.setJumlahMissing(10);

        // Objek 3: Constructor lengkap
        Dataset dataset3 = new Dataset("Iris", 150, 5, 0);

        // Simpan ketiganya dalam array
        Dataset[] daftarDataset = { dataset1, dataset2, dataset3 };

        // Cetak laporannya dengan perulangan
        for (Dataset d : daftarDataset) {
            LaporanDataset.cetak(d);
        }

        // Tampilkan total dataset yang dibuat di akhir
        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project4Unguided ---
=== Laporan Dataset ===
Nama          : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing       : 866 sel (8,10%)
Status        : Perlu dibersihkan

=== Laporan Dataset ===
Nama          : Wine Quality
Jumlah Baris : 4898
Jumlah Kolom : 12
Missing       : 10 sel (0,02%)
Status        : Bersih

=== Laporan Dataset ===
Nama          : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing       : 0 sel (0,00%)
Status        : Bersih

Total dataset dibuat : 3
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.711 s
Finished at: 2026-10-11T20:05:55+07:00
------------------------------------------------------------------------
```

Penjelasan :
Program Sistem Pencatatan Dataset ini dirancang dengan menerapkan konsep enkapsulasi dan pembagian package agar struktur kode lebih terstruktur, aman, dan mudah dikelola. Pada package model, terdapat class Dataset yang berfungsi sebagai representasi objek data dengan menerapkan prinsip enkapsulasi, di mana seluruh atribut utamanya seperti nama, jumlahBaris, jumlahKolom, dan jumlahMissing dideklarasikan menggunakan akses modifier private. Class ini dilengkapi dengan tiga bentuk constructor (overloading), konstanta BATAS_MISSING sebesar 5.0%, serta variabel statis untuk menghitung total objek yang berhasil dibuat. Selain itu, class Dataset juga menyediakan fungsi logika untuk menghitung persentase missing value secara otomatis serta menentukan apakah dataset tersebut memerlukan proses pembersihan (cleaning) berdasarkan batas yang telah ditentukan.

Selanjutnya, pada package laporan, terdapat class LaporanDataset yang bertanggung jawab untuk menangani proses presentasi dan keluaran data ke konsol. Class ini memiliki sebuah method statis bernama cetak yang menerima parameter berupa objek Dataset, kemudian mengekstrak informasi detail seperti nama, dimensi baris dan kolom, jumlah serta persentase sel kosong, hingga status kelayakan dataset menggunakan format tampilan yang rapi dan konsisten. Pemisahan tugas antara class Dataset sebagai pengelola data dan class LaporanDataset sebagai pencetak laporan ini menerapkan prinsip separation of concerns, sehingga kode menjadi lebih modular dan terhindar dari ketergantungan fungsional yang berlebihan.

Bagian akhir dari program ini diatur di dalam package main melalui class Main, yang bertindak sebagai titik eksekusi utama aplikasi. Di dalam class ini, tiga objek Dataset diinisialisasi menggunakan teknik pembuatan yang berbeda-beda sesuai dengan ketentuan, yakni menggunakan constructor tanpa parameter yang kemudian diisi melalui setter, constructor dengan satu parameter nama, serta constructor lengkap. Ketiga objek tersebut kemudian disimpan ke dalam struktur data array agar dapat diproses secara efisien menggunakan perulangan untuk mencetak laporannya melalui class LaporanDataset, serta diakhiri dengan menampilkan informasi total keseluruhan objek dataset yang telah berhasil diciptakan.

D. Kesimpulan

Berdasarkan praktikum Modul 4 mengenai Enkapsulasi, dapat disimpulkan bahwa:
Modifier (seperti private, public, dan static) berperan penting dalam mengatur tingkat aksesibilitas serta hak akses pada kelas, atribut, dan method. Penggunaan akses modifier private pada atribut mendukung konsep data hiding (penyembunyian data) untuk menjaga keamanan integritas data dari akses luar yang tidak sah.

Constructor merupakan method khusus yang digunakan untuk menginisialisasi nilai awal pada sebuah objek saat pertama kali dibuat. Konsep constructor overloading memungkinkan sebuah kelas memiliki lebih dari satu constructor dengan parameter yang berbeda-beda, sehingga memberikan fleksibilitas dalam instansiasi objek.

Package berfungsi untuk mengelompokkan kelas-kelas yang saling berkaitan ke dalam direktori tertentu agar struktur kode program menjadi lebih terstruktur, modular, dan terhindar dari konflik penamaan, serta memerlukan keyword import apabila diakses antar package yang berbeda.